PROGRAMAÇÃO DISTRIBUÍDA

Ficha de exercícios

REVISÃO – SOCKETS WINDOWS

1. Desenvolva uma aplicação cliente/servidor Win32, baseada no protocolo UDP, que permita obter a hora atual no sistema que alberga a aplicação servidora. Os clientes recebem a localização do servidor através da linha de comando e terminam depois do resultado ser apresentado na saída *standard*. Antes de serem apresentadas, as horas obtidas (pode recorrer à função *GetLocalTime*) devem ser acertadas em função dos respetivos tempos de resposta.
2. Desenvolva uma aplicação cliente/servidor Win32, baseada no protocolo TCP, com funcionalidades semelhantes às da aplicação realizada no âmbito do exercício 1. O servidor é do tipo iterativo.
3. Transforme o servidor desenvolvido no exercício 2 num servidor concorrente.
4. Desenvolva uma aplicação cliente/servidor Win32, baseada no protocolo TCP, que permita obter um ficheiro armazenado no servidor. A aplicação cliente deve ser lançada passando, na linha de comando, a localização do servidor, o nome do ficheiro pretendido e o nome com que este ficará guardado localmente. O servidor deve ser lançado passando, na linha de comando, o porto de escuta. Deve ser possível transferir ficheiros com qualquer dimensão. Para o efeito, defina um tamanho máximo para os blocos transferidos (por exemplo, MAX_DATA = 512 bytes), sendo a conclusão de transferência de um determinado ficheiro assinalada através do encerramento da ligação TCP pelo servidor. Qualquer problema que surja durante a transferência de um ficheiro, incluindo situações de *timeout*, leva a que esta seja abortada.

SOCKETS JAVA

5. Desenvolva uma aplicação cliente/servidor em Java, baseada no protocolo UDP, compatível com a versão Win32 realizada no exercício 1.
6. Desenvolva uma aplicação cliente/servidor em Java, baseada no protocolo UDP, que permita obter um ficheiro armazenado no servidor. A aplicação cliente deve ser lançada passando na linha de comando a localização do servidor, o nome do ficheiro pretendido e a diretoria local onde a cópia obtida deve ser armazenada. O servidor deve ser lançado passando, na linha de comando, o porto de escuta e a diretoria local onde se encontram os ficheiros suscetíveis de serem carregados pelos clientes. O servidor deve apenas permitir o acesso a ficheiros localizados na diretoria indicada ou subdiretorias. Deve ser possível transferir ficheiros com qualquer dimensão. Para o efeito, defina um tamanho máximo para os blocos transferidos (por exemplo, MAX_DATA = 4000 bytes), correspondendo o último bloco de um ficheiro a um datagrama UDP com conteúdo de tamanho igual a zero bytes. O cliente começa por enviar ao servidor um datagrama com conteúdo correspondente ao nome do ficheiro pretendido. Qualquer problema que surja durante a transferência de um ficheiro, incluindo situações de *timeout*, leva a que esta seja abortada.

7. Desenvolva uma aplicação cliente/servidor em Java, baseada no protocolo TCP, compatível com a versão Win32 realizada no exercício 2.
8. Desenvolva uma aplicação cliente/servidor em Java, baseada no protocolo TCP, com funcionalidades semelhantes às da aplicação realizada no âmbito do exercício 6. Neste caso, o servidor assinala a conclusão de transferência de um ficheiro encerrando a respetiva ligação TCP.
9. Altere as aplicações cliente/servidor UDP e TCP desenvolvidas nos exercícios 5, 6, 7 e 8 de modo a que sejam trocados objetos serializados (do tipo *String*) em vez de cadeias de caracteres. Analise o tráfego gerado e compare-o com aquele que é gerado recorrendo às versões originais.
10. Transforme os servidores TCP desenvolvidos em Java, no âmbito dos exercícios 7 e 8, em servidores concorrentes.
11. Desenvolva, na linguagem Java, uma aplicação (*peer-to-peer*) elementar que permita a troca de mensagens no seio de um grupo de utilizadores através da utilização de endereços IP do tipo *multicast* (i.e., pertencentes à classe D). As aplicações são lançadas passando, na linha de comando, o nome do utilizador, o endereço de grupo e o porto de escuta pretendidos. Estas ficam continuamente à espera de mensagens introduzidas na entrada standard, as quais são enviadas para o grupo de conversa, e de mensagens recebidas via rede, as quais são mostradas na saída standard. As mensagens são trocadas sob a forma de cadeias de caracteres. Uma mensagem recebida com a sequência "LIST" faz com que a aplicação reenvie o nome do utilizador local à origem.
12. Altere a aplicação desenvolvida no âmbito da pergunta anterior de modo a que sejam trocadas instância da classe *Msg*, sendo esta constituída pelos atributos *protected String* ***nickname*** e *protected String* ***msg***, bem como pelos métodos *public* ***Msg****(String nickname, String msg)*, *public String* ***getNickname****()* e *public String* ***getMsg****()*. A resposta a uma sequência de caracteres "LIST" é dada através de um objeto serializado to tipo *String*.
13. Desenvolva, na linguagem Java, um servidor TCP concorrente do tipo *proxy*, ou seja, que faça a ponte entre clientes TCP e um servidor TCP específico. A localização deste último é fornecida ao *proxy* através da linha de comando.
14. a) Desenvolva, na linguagem Java, uma aplicação cliente/servidor TCP que permita usar uma rede de computadores para calcular o valor de Π por integração numérica de um modo paralelo. Na equação seguinte, *n* corresponde ao número de intervalos e x_i à abcissa do centro do i-ésimo intervalo.

$$\Pi = \int_0^1 \frac{4}{1 + x^2}\, dx \cong \frac{1}{n}\sum_{i=1}^{n} \frac{4}{1 + x_i^2}$$

O objetivo é transformar um conjunto (*cluster*) de computadores numa espécie de computador paralelo. A aplicação cliente (i.e., *master*) recebe, através da linha de comando, o nome de um ficheiro de texto onde se encontram definidas as localizações dos servidores (i.e., *workers*), bem como o número de intervalos pretendidos (i.e., a variável *n*). Ao arrancar, o *master* começa por enviar, a cada *worker*, um objeto serializado com os seguintes atributos encapsulados: número de intervalos, número de *workers* e respetivo índice (i.e., se é o 1º *worker*, o 2º *worker*, etc.). Cada *worker* calcula a sua parte do valor de Π, com base nos dados que lhe foram fornecidos, e envia-a ao *master* através da respetiva ligação TCP, sob a forma de uma instância da classe

Double. O *master*, depois de distribuir as tarefas, aguarda pela receção de todos os resultados parciais, mostra-os na saída standard, soma-os e apresenta o resultado final (i.e., o valor aproximado de Π obtido por integração numérica).

b) Desenvolva, em linguagem Java, uma aplicação distribuída sobre TCP que permita criar uma cópia, em escala de cinza, de uma imagem a cores do tipo JPG, através de uma abordagem de processamento paralelo. A aplicação é constituída por um *master* e por um ou vários *workers*. Considere, igualmente, no âmbito deste exercício, que existe um conjunto de ficheiros JPG replicados em todas as máquinas onde se encontram a correr os *workers* e o *master*.

A aplicação *master* recebe, através da linha de comando, o nome de um ficheiro de texto onde se encontram definidas as localizações dos *workers*, o nome do ficheiro JPG a processar e o nome da cópia em escala de cinza a produzir. Quando arranca, o *master* estabelece uma ligação TCP com cada *worker* e envia, a cada um, um objeto serializado com os seguintes atributos: nome do ficheiro JPG a converter para uma escala de cinza, número de *workers* e respetivo índice (i.e., se é o 1º *worker*, o 2º *worker*, etc.). Cada *worker* determina o novo valor dos pixéis na sua parte da imagem, acedendo a réplica original disponível localmente, e envia-os ao *master* através da respetiva ligação TCP, sob a forma de uma instância da classe `ArrayList`. Se ocorrer algum problema durante a execução da sua tarefa, um *worker* deve transmitir um objeto do tipo `Exception` (ou derivado) em vez de um `ArrayList`. O *master*, depois de distribuir as tarefas, aguarda pela receção de todos os resultados parciais, compõe a imagem em escala de cinza com base nestes e guarda-a localmente.

O código seguinte exemplifica de que forma é possível criar uma cópia em escala de cinza de uma imagem JPG:

```
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import javax.imageio.ImageIO;

public class ImageToGrayScale {

    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: java ImageToGrayscale <input_image.jpg>
                                                    <output_image.jpg>");
            return;
        }

        String inputImagePath = args[0];
        String outputImagePath = args[1];

        try {
            BufferedImage image = ImageIO.read(new File(inputImagePath));

            for (int y = 0; y < image.getHeight(); y++) {
                for (int x = 0; x < image.getWidth(); x++) {
                    int rgb = image.getRGB(x, y);
                    int r = (rgb >> 16) & 0xFF;
                    int g = (rgb >> 8) & 0xFF;
                    int b = (rgb) & 0xFF;

                    // Average intensity (grayscale value)
```

```
                    int gray = (r + g + b) / 3;

                    image.setRGB(x, y, (gray << 16) | (gray << 8) | gray);
                }
            }

            ImageIO.write(image, "jpg", new File(outputImagePath));

        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

15. a) Desenvolva uma variação da aplicação desenvolvida no exercício 14 para que o *master* obtenha a lista de *workers* (endereços IP e portos de escuta TCP) através de uma base de dados MySQL. O endereço do servidor de base de dados, o nome da base de dados e as credenciais de acesso são fornecidos através da linha de comando.

    b) Faça uma variação da aplicação desenvolvida no exercício 14 para que o *master* descubra automaticamente os endereços IP e os portos de escuta dos *workers* presentes na rede local enviando um único *datagrama* UDP para o endereço de difusão 255.255.255.255 e porto 5001.

    c) Faça uma variação da aplicação desenvolvida no exercício 14 em que o *master* comece por entrar num ciclo de aceitação de ligações de *workers* depois de ter enviado um único *datagrama* UDP, que transporta a representação serializada do valor do porto de escuta associado ao seu socket TCP, para o endereço 255.255.255.255 e porto 5001. O porto associado ao *socket* de escuta do servidor deve ser atribuído de forma automática.

CLIENTES HTTP EM JAVA E FORMATO JSON

16. Desenvolva, em Java, aplicações cliente para as URI abaixo, recorrendo a classes dos *packages* `java.net`, `com.google.gson` e `javax.json`. Comece por analisar a estrutura das respostas obtidas em formato JSON (chaves e valores). Para o efeito, é suficiente usar um browser qualquer. Explore a parte variável das URI caso exista, por exemplo, através de argumentos na linha de comandos das aplicações.

    a) https://catfact.ninja/fact.

    Sugestão de *output* esperado:

```
Fact: Cats that live together sometimes rub each others heads to show that they
 have no intention of fighting. Young cats do this more often, especially when
they are excited.
```

    b) http://universities.hipolabs.com/search?country=x, sendo x o nome de um país.

    Sugestão de output esperado:

```
  - European Institute of Education (Malta)
  - International Tourism Institute (Malta)
  - University of Malta (Malta)
```

c) https://potterapi-fedeperin.vercel.app/en/characters?index=x, sendo x o índice da personagem.

Sugestão de *ouput* esperado:

```
index: 1
fullName: Hermione Jean Granger
birthdate: Sep 19, 1979
hogwartsHouse: Gryffindor
interpretedBy: Emma Watson
image: https://raw.githubusercontent.com/fedeperin/potterapi/main/public/images/characters/hermione_granger.png
children: [Rose Granger-Weasley, Hugo Granger-Weasley]
```

d) https://potterapi-fedeperin.vercel.app/en/characters, com o objetivo de obter e guardar numa diretoria local, especificada através de um argumento na linha de comandos, as imagens das personagens do filme. Não se deve recorrer a classes dos *packages* `java.awt.image.BufferedImage` nem `javax.imageio.ImageIO`.

Sugestão de resultado esperado:

Albus-Dumbledore.jpg	21/10/2024 16:15	JPG File	320 KB
Arthur-Weasley.jpg	21/10/2024 16:02	JPG File	288 KB
Bellatrix-Lestrange.jpg	21/10/2024 16:02	JPG File	242 KB
Bill-Weasley.jpg	21/10/2024 16:02	JPG File	241 KB
Cedric-Diggory.jpg	21/10/2024 16:15	JPG File	339 KB
Charlie-Weasley.jpg	21/10/2024 16:02	JPG File	243 KB
Draco-Malfoy.jpg	21/10/2024 16:02	JPG File	185 KB
Fred-Weasley.jpg	21/10/2024 16:02	JPG File	312 KB
George-Weasley.jpg	21/10/2024 16:02	JPG File	207 KB
Ginny-Weasley.jpg	21/10/2024 16:02	JPG File	233 KB
Harry-Potter.jpg	21/10/2024 16:15	JPG File	171 KB
Hermione-Granger.jpg	21/10/2024 16:15	JPG File	362 KB
James-Potter.jpg	21/10/2024 16:15	JPG File	261 KB

API REST (SPRING BOOT)

17. Desenvolva, recorrendo à plataforma Spring Boot, uma API REST que permita obter informações sobre personagens dos filmes Harry Potter. Para o efeito, é fornecido um ficheiro JSON que inclui a descrição das personagens (*array* JSON) e um ficheiro de imagem relativo a cada uma. Os *endpoints* suportados pelo serviço Web são os seguintes:

    - `GET /movies/harry-potter/characters` (parâmetro opcional: `index`) ⇒ devolve, em formato JSON, a descrição das personagens ou, caso seja especificado um índice, da respetiva personagem. Como sugestão, recorra à classe `com.fasterxml.jackson.databind.ObjectMapper` para construir uma lista de personagens a partir do ficheiro JSON fornecido;
    - `GET /movies/harry-potter/characters/images` (parâmetro opcional: index) ⇒ devolve, em formato JSON, a lista dos nomes dos ficheiros de imagem

existentes ou, caso seja especificado um índice, o nome do respetivo ficheiro em *plain text*;

- `GET /movies/harry-potter/characters/images/{file-name}` ⇒ devolve o conteúdo do ficheiro especificado na URI. Como sugestão, recorra à classe `org.springframework.core.io.UrlResource` para obter e colocar na resposta HTTP os bytes do ficheiro;
- `PUT /movies/harry-potter/characters/images/{file-name}` ⇒ cria um ficheiro do lado do servidor com o nome especificado na URI e com conteúdo igual ao corpo do pedido. A operação deve falhar se já existir um ficheiro com o mesmo nome ou se o campo `Content-Type` do cabeçalho for diferente de "image/png".
- `GET /movies/harry-potter/characters/images/{file-name}/length` ⇒ devolve, em *plain text*, o tamanho do ficheiro especificado.

Devem ser devolvidos códigos de resposta HTTP apropriados às diversas situações que possam ocorrer.

18. Inclua no serviço Web desenvolvido na alínea anterior um mecanismo de autenticação com as seguintes características:
    - *Login* baseado em autenticação básica HTTP, através do *endpoint* `GET /login`, com devolução de um *token* JWT;
    - Acesso sem autenticação aos *endpoints* `GET /movies/harry-potter/characters` e `GET /movies/harry-potter/characters/images/{file-name}/length`;
    - Acesso com autenticação JWT aos restantes *endpoints*.

JAVA RMI

19. Desenvolva, em Java RMI, uma aplicação cliente/servidor que permita obter a hora atual no sistema que alberga o serviço remoto. Os clientes recebem a localização do RMI *registry* onde se encontra registado o serviço através da linha de comando e terminam depois do resultado ser apresentado na saída *standard*. O serviço remoto e o *registry* onde este se encontra registado sob o nome ***timeserver*** correm na mesma máquina. O resultado é fornecido através de uma instância da classe ***Hora***, sendo esta constituída pelos atributos *static final long* ***serialVersionUID*** *= 1L*, *protected int* ***horas***, *protected int* ***minutos*** e *protected int* ***segundos***, bem como pelos métodos públicos ***Hora****(int hora, int minuto, int segundos)*, *getters* e ***toString***. O serviço remoto implementa a interface remota ***RemoteTimeInterface***, sendo esta constituída apenas pelo método ***Hora getHora()***.

20. Desenvolva uma aplicação cliente/servidor em Java RMI que permita obter um ficheiro armazenado no computador que aloja o serviço remoto. A aplicação cliente deve ser lançada passando, na linha de comando, a localização do RMI *registry* onde se encontra registado o serviço bem como o nome do ficheiro pretendido. O serviço remoto deve ser lançado passando, na linha de comando, a diretoria local onde se encontram os ficheiros suscetíveis de serem carregados pelos clientes. O serviço remoto e o *registry* onde este se encontra registado correm na mesma máquina. Deve ser possível transferir ficheiros com qualquer dimensão. Sendo assim, a interface remota do serviço deve possibilitar a obtenção de blocos em vez de ficheiros

completos (e.g., *byte []* ***getFileChunk****(String fileName, long offset)*). Para o efeito, defina um tamanho máximo para os blocos solicitados (por exemplo, MAX_CHUNK_LENGTH = 512 bytes). Qualquer problema que surja durante a obtenção de um ficheiro leva a que esta operação seja cancelada.

21. Desenvolva, em Java RMI, uma versão do exercício 20 que permita aos clientes solicitarem um ficheiro de tamanho qualquer invocando uma única vez um determinado método (e.g., *void* ***getFile****(String fileName, GetRemoteFileClientInterface cliRef) throws java.io.IOException*) no serviço remoto. Para o efeito, deve recorrer a um mecanismo de *callback*. Em concreto, um cliente, depois de abrir o ficheiro local para escrita, invoca o método *getFile* na interface remota passando-lhe o nome do ficheiro pretendido e a referência para a interface remota que ele próprio inclui. É o método *getFile* do serviço remoto que, para cada bloco do ficheiro pretendido, invoca o método *writeFileChunk* na interface remota do cliente. Este método possui apenas um *array* de *bytes* e um inteiro como argumentos (e.g., *void* ***writeFileChunk****(byte [] fileChunk, int nbytes) throws java.io.IOException*).

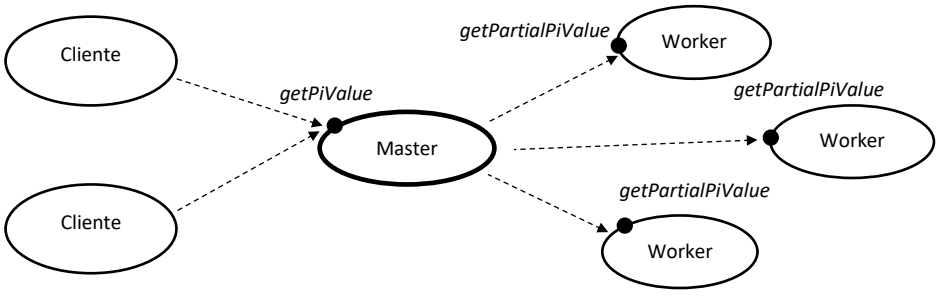
22. Desenvolva, em Java RMI e com recurso a *callbacks*, um observador remoto que é, de um modo assíncrono, notificado das operações realizadas pelo serviço remoto desenvolvido no âmbito do exercício anterior. Este deve incluir a interface remota ***GetRemoteFileObserverInterface*** constituída apenas pelo método ***notifyNewOperationConcluded****(String description)*.

23. Desenvolva, em Java RMI, uma aplicação que permita calcular o valor de Π de forma paralela num *cluster* de computadores. A solução deve incluir três tipos de aplicações:

    a. os ***workers***, destinados a participar no cálculo do valor de Π e que incluem um serviço RMI: interface remota ***PartialPiValueInterface*** constituída pelo método único ***getPartialPiValue****(long nIntervals, int nWorkers, int index)*;

    b. o ***master***, que inclui um serviço RMI: interface remota ***PiValueInterface*** constituída pelo método único *double* ***getPiValue****(long nIntervals)*;

    c. os **clientes** que invocam o método *getPiValue* no serviço remoto do ***master*** para obter o valor de Π.



O método *getPiValue* recorre a *threads* adicionais para poder invocar o método *getPartialPiValue* nos vários *workers* em simultâneo (ver sugestões mais abaixo).

Cada *worker* regista o seu serviço RMI no *registry* local sob o nome ***piWorker***. O *master* faz o mesmo recorrendo ao nome ***piFrontEnd***.

O *master* recebe, através da linha de comando, o nome de um ficheiro de texto que inclui as localizações dos RMI *registries* onde se encontram registados os serviços remotos dos *workers*.

Sugestões:

- Depois de processar o ficheiro com os endereços IP/nomes dos *workers*, o *master* pode armazenar as referências remotas correspondentes numa lista ou conjunto;
- Como o método *getPartialPiValue* de cada *worker* é bloqueante, o método *getPiValue* pode recorrer a *threads* adicionais para que se obtenha o efeito de concorrência pretendido. A referência remota de um *worker* pode ser passada como argumento no construtor da respetiva *thread* e o resultado obtido pode ser armazenado num dos membros desta *thread*.

24. Desenvolva, em Java RMI e com recurso a *callbacks*, um observador remoto que é, de um modo assíncrono, notificado dos cálculos efetuados pelo master desenvolvido no âmbito do exercício 23. Este deve incluir a interface remota ***RemotePiObserver*** constituída apenas pelo método ***notifyNewCalculation****(CalculationDetails cd)*. A classe ***CalculationDetails*** inclui os atributos *nIntervals* e *result*, os respetivos *getters* e um construtor que recebe os valores dos dois atributos.
25. Desenvolva uma variante da aplicação distribuída especificada no exercício 23 em que são os *workers* que se registam no *master*.
26. Desenvolva, em Java RMI, uma aplicação distribuída com a mesma funcionalidade da aplicação especificada no exercício 14-b).